

COMP3320/6464 High-Performance Scientific Computing Cloth Simulation

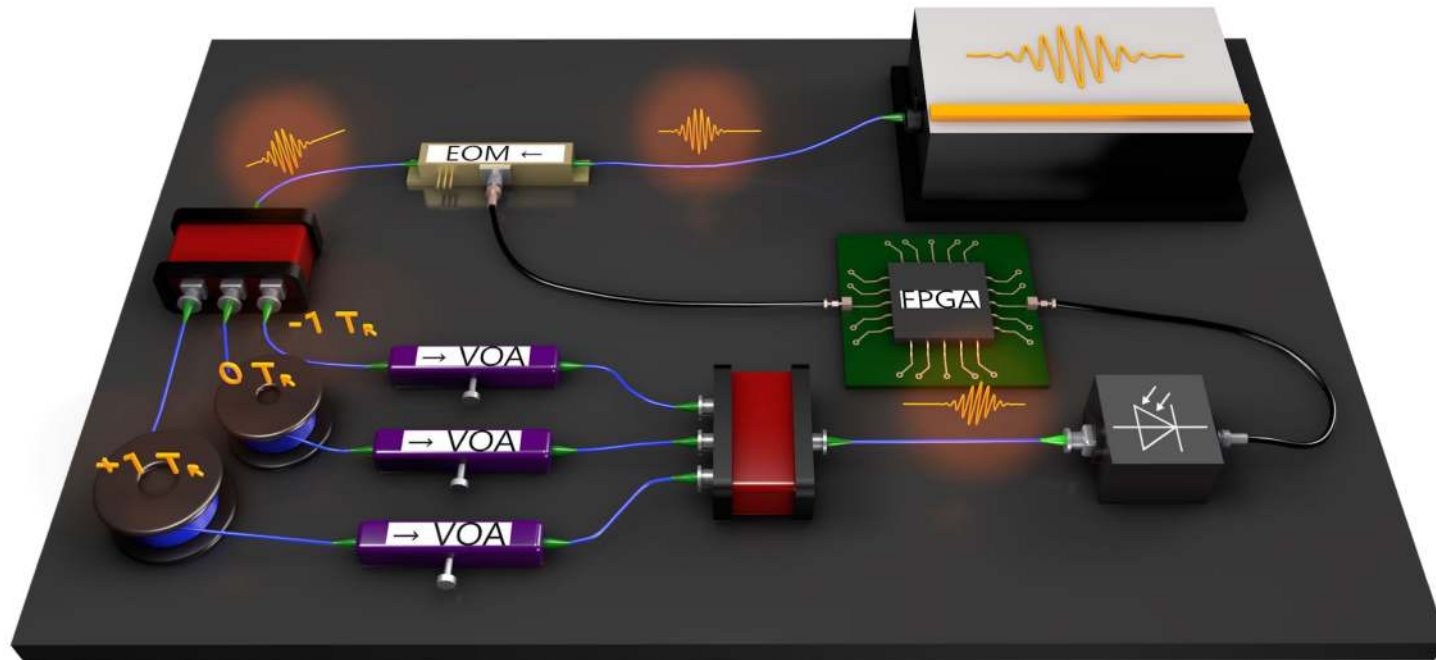
Dr Haibo Zhang
School of Computing
Australian National University



Australian
National
University

ANU College of
Engineering & Computer Science

Photonic computing uses light (photons) instead of electrons to perform computations in the optical domain.



[Gordon H. Y. Li et al. light: science & applications](#)

Photonic computing uses light (photons) instead of electrons to perform computations in the optical domain.

<https://haibozhang-web.github.io/img/PhotonicComputing.mp4>

Changes to lab schedule

- **No lab in this week**

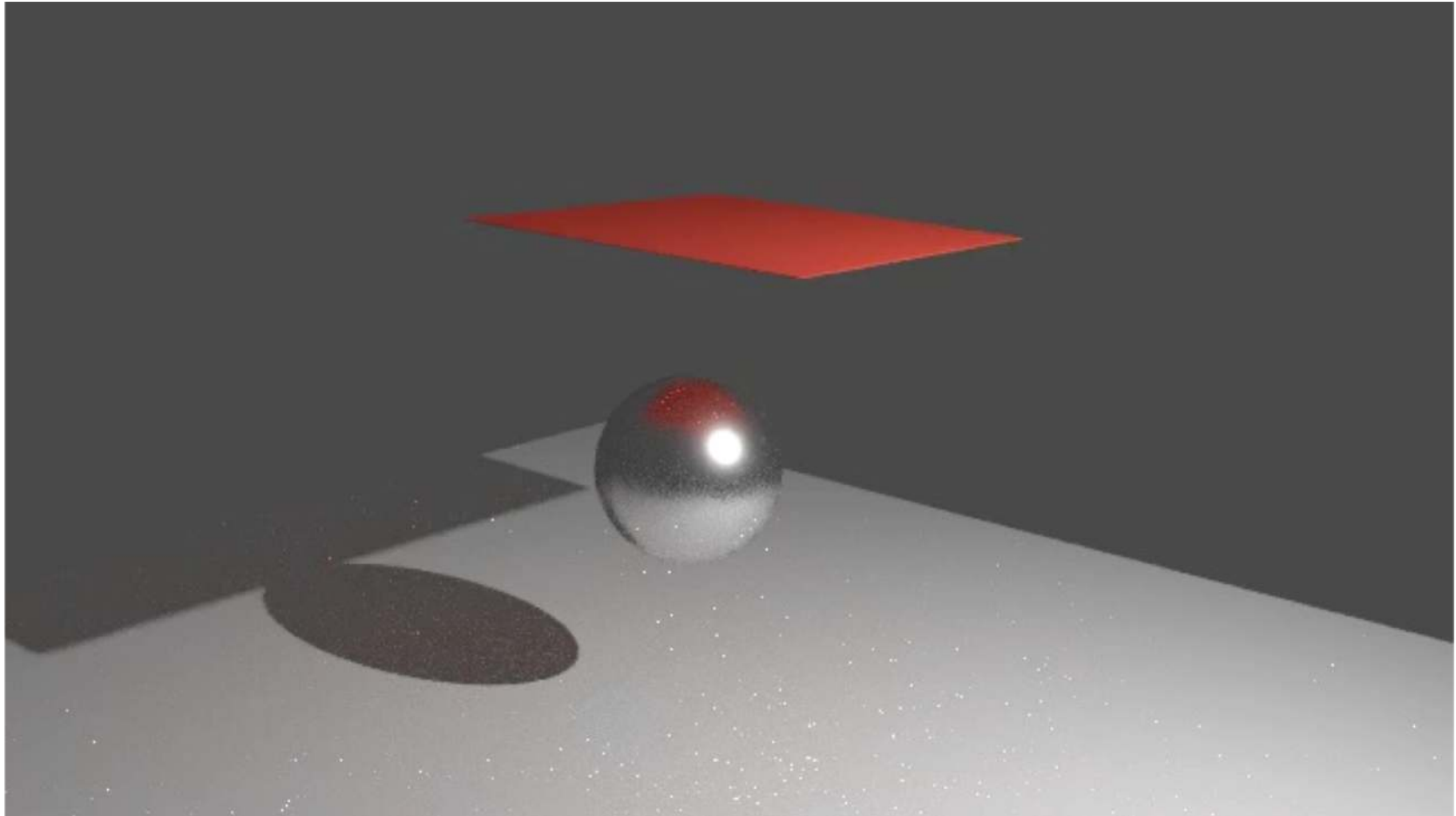
Week 9 (Sept 30)	Lab 4: OpenMP
Week 10 (Oct 7)	Assignment 2 drop-in session Lab 5: Advanced OpenMP
Week 11 (Oct 14)	Lab 6: OpenMP SIMD

This week

- Monday: assignment 2
- Tuesday: thread memory model
thread synchronization

Assignment 2 – Cloth Simulation

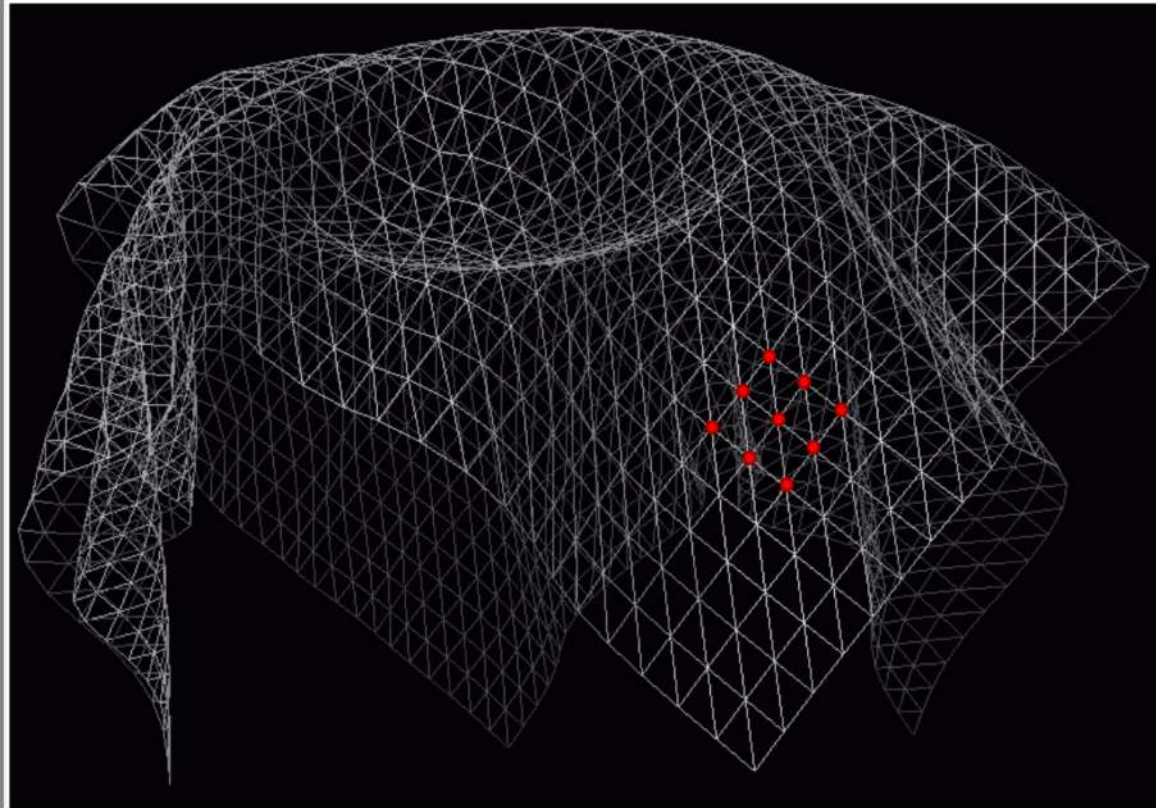
Use computer graphics and animation to model the behaviour of clothes as they move, fold, and interact with their environment.



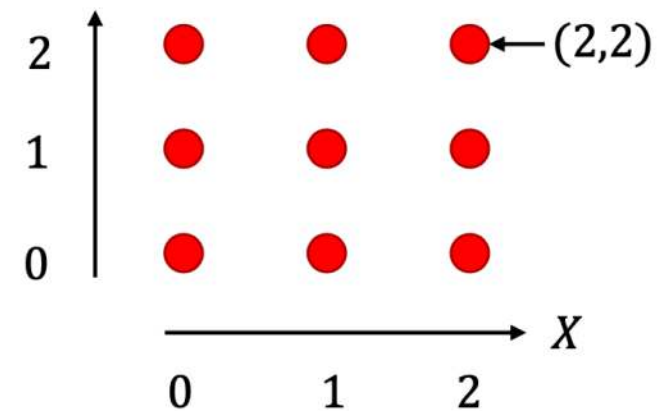
Source: https://en.wikipedia.org/wiki/File:Cloth_Simulation.ogv

What are the applications of cloth simulation?

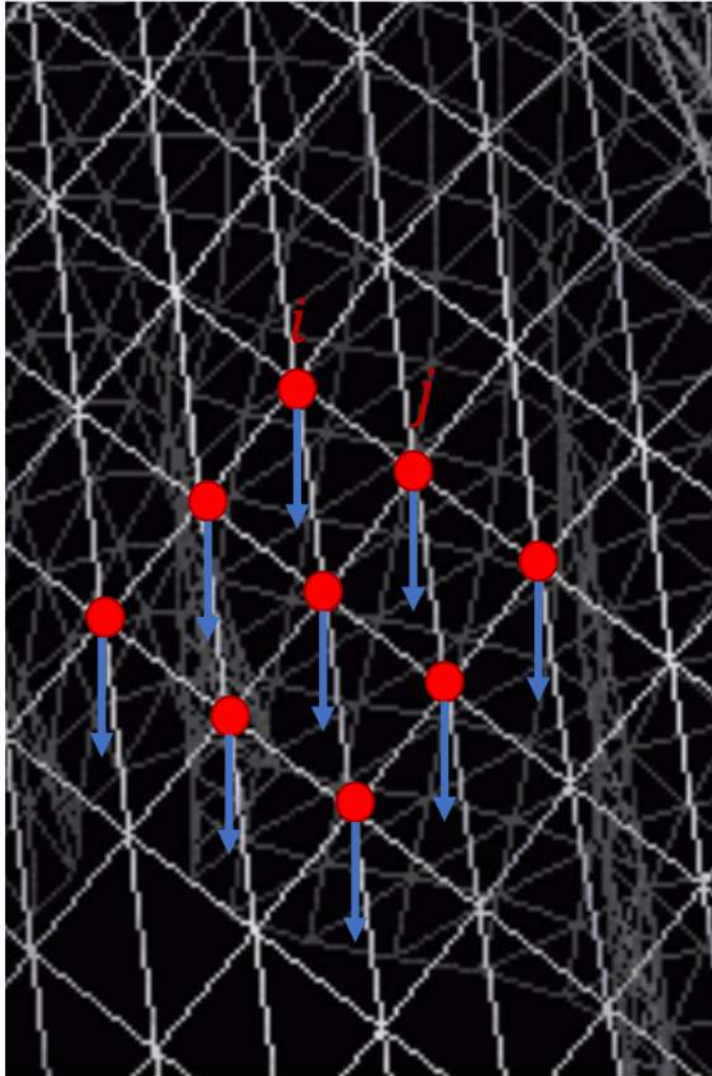
The physics of a cloth simulation



- The cloth is represented using a 2D $N \times N$ mesh of nodes (red dots)
- Each node has its own cartesian coordinates (x, y, z)
- Each node has also mesh coordinates (X, Y)



The physics of a cloth simulation



- Each node is subjected to two forces: gravity and elastic force
- Gravity – each node is associated with a mass m_{node} and subject to the gravity

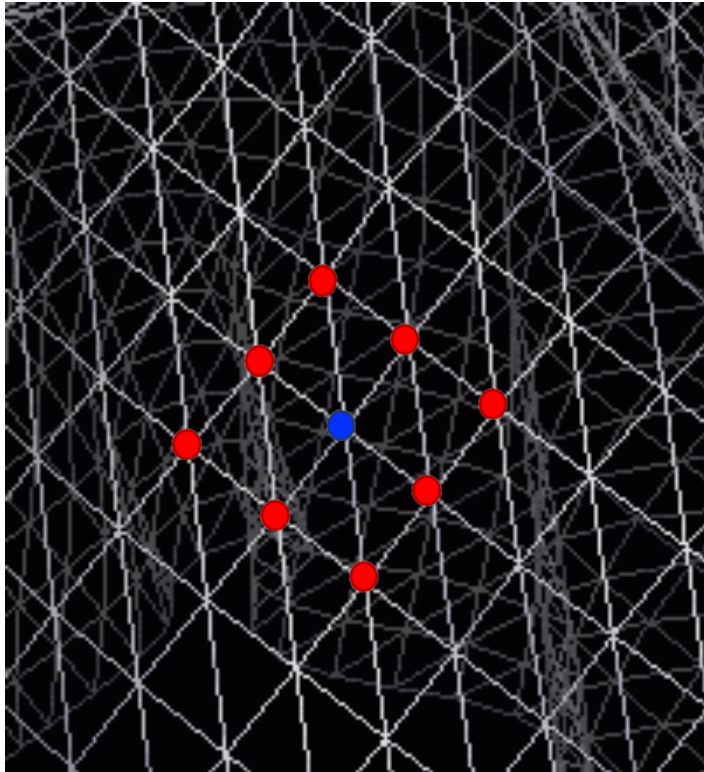
$$\vec{G} = m_{node} \vec{g}$$

- Elastic force – each pair of nodes interacts with a restoring force given by Hooke's law

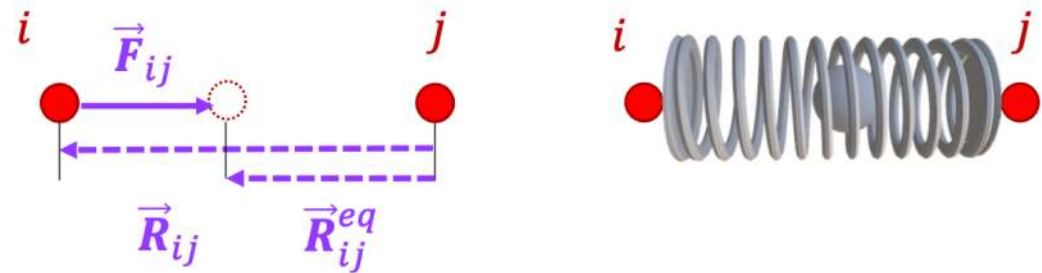
$$\vec{F}_{ij} = -k (\vec{R}_{ij} - \vec{R}_{ij}^{eq})$$

- Here “ i ” refers to a node, with mesh coordinates (X_i, Y_i)

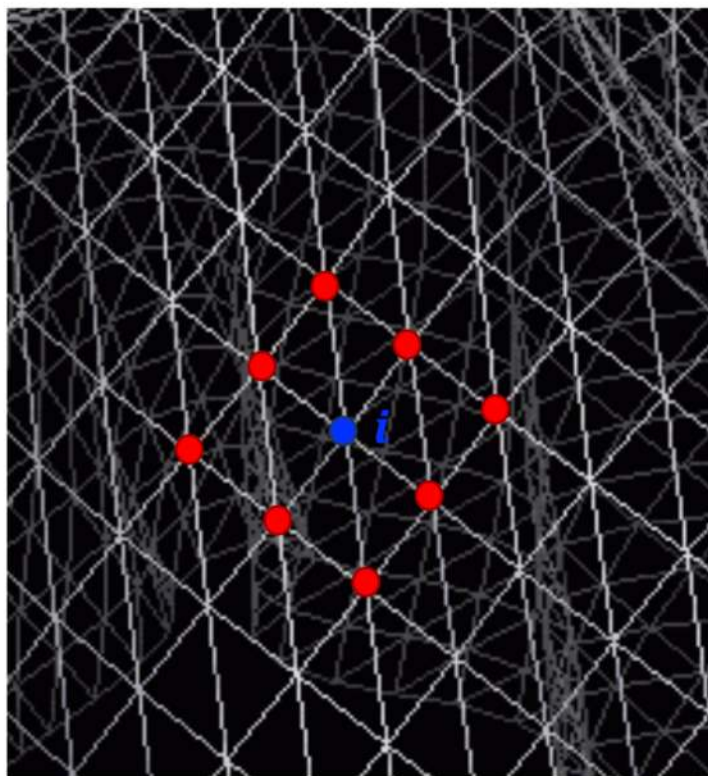
- Elastic force – each pair of nodes interacts with a restoring force given by Hookes law



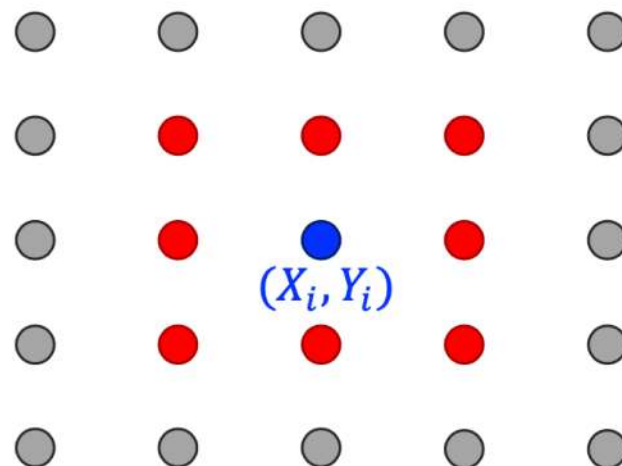
$$\vec{F}_{ij} = -k (\vec{R}_{ij} - \vec{R}_{ij}^{eq})$$



- The force acts as if there was a spring between i and j
- k is the force constant determining how stiff the spring is
- $\vec{R}_{ij}$ is the Euclidean distance between node j and node i
- $\vec{R}_{ij}^{eq}$ is the equilibrium distance between node j and node i



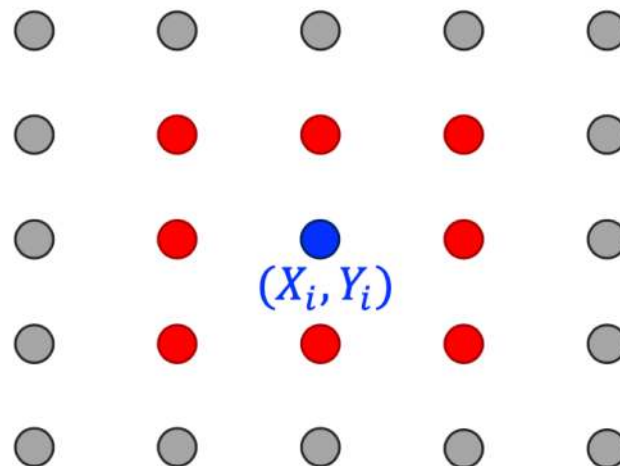
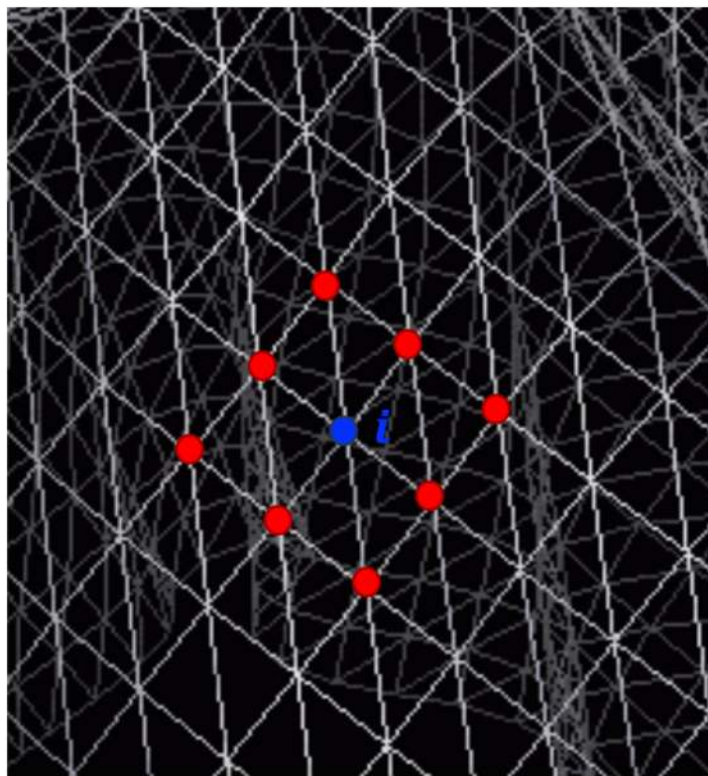
- We assume that a node i with mesh coordinates (X_i, Y_i) interacts through elastic force with all neighbors within mesh coordinates $(X_i - \delta, Y_i - \delta)$ and $(X_i + \delta, Y_i + \delta)$
- For $\delta = 1$ the blu node interacts with all the red nodes (for a total of 8)
- For $\delta = 2$ the blu node interacts with all the red and the grey nodes (for a total of 24)



- What is the total force acting on a single node “ i ”?

$$\vec{F}_i = \vec{G} + \sum_{\substack{j \neq i: \\ X_i - \delta \leq X_j \leq X_i + \delta \\ Y_i - \delta \leq Y_j \leq Y_i + \delta}} \vec{F}_{ij}$$

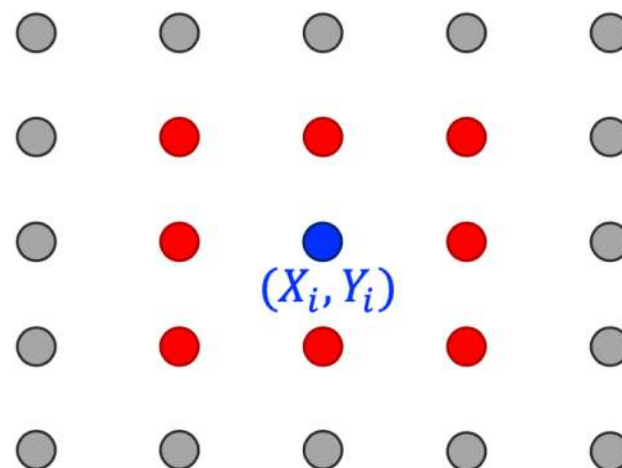
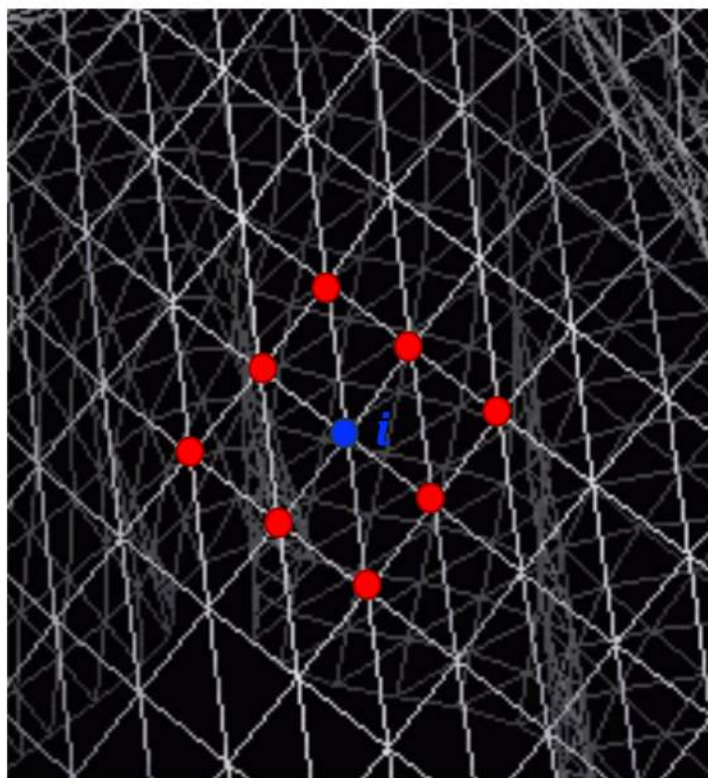
- Assuming $\delta \ll N$, how does the computation of the forces for all nodes scale (time complexity)?



- What is the total force acting on a single node “ i ”?

$$\vec{F}_i = \vec{G} + \sum_{\substack{j \neq i: \\ X_i - \delta \leq X_j \leq X_i + \delta \\ Y_i - \delta \leq Y_j \leq Y_i + \delta}} \vec{F}_{ij}$$

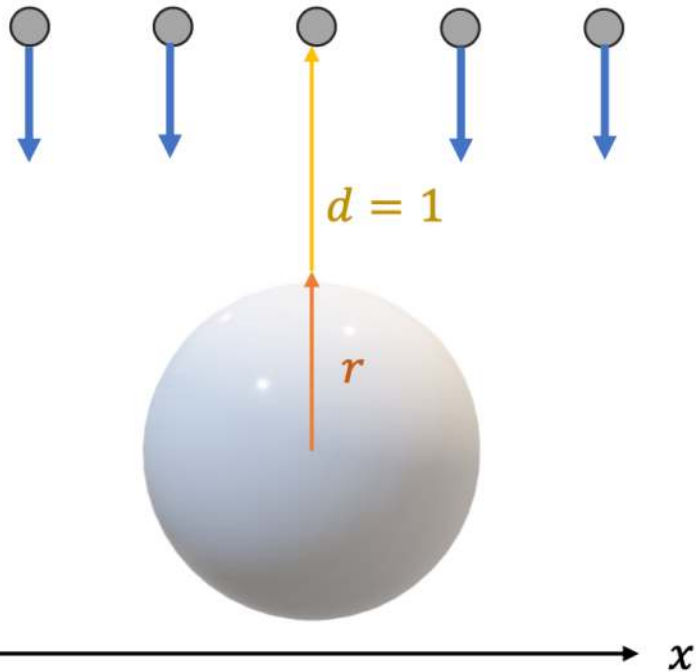
- How does the computation of the forces for all nodes scale (time complexity)? As $O(M)$ where M is the total number of nodes



The problem setting



$t = 0$

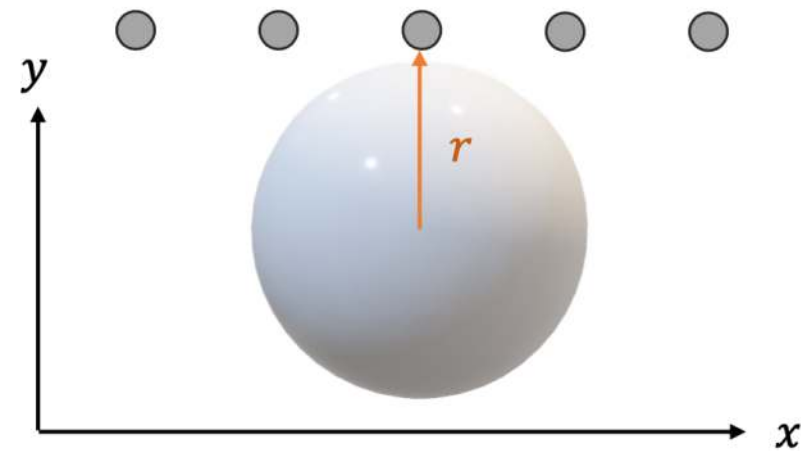


- The cloth is initially positioned in the xz plane.
- Positioned below the cloth there is a ball of radius r .
- The centre of the cloth is located $1+r$ units above the centre of the ball.
- The cloth is allowed to fall under gravity until it collides with the ball.

The problem setting



$t = t_1$



- The cloth is allowed to fall under gravity until it collides with the ball ($t=t_1$)
- You follow the motion of the nodes using the same velocity verlet algorithm that you used for the MD

$$y_i(t_k) = y_i(t_{k-1}) + v_i^y(t_{k-1})\Delta t + \frac{1}{2} \frac{F_i^y(t_{k-1})}{m_{node}} \Delta t^2$$

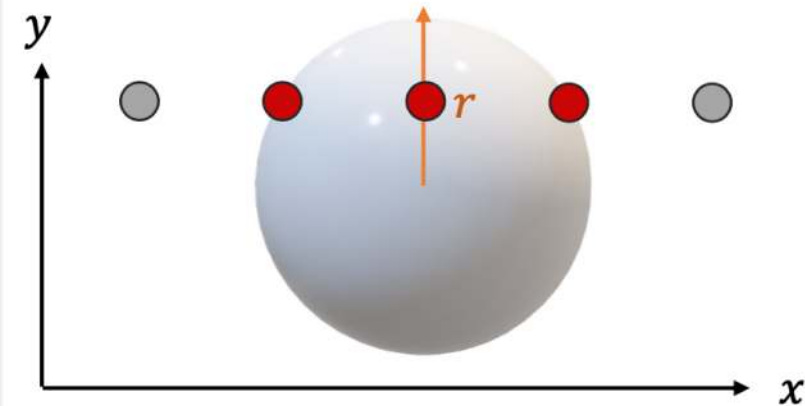
$$v_i^y(t_k) = v_i^y(t_{k-1}) + \frac{1}{2} \frac{F_i^y(t_k) + F_i^y(t_{k-1})}{m_{node}} \Delta t$$

- The difference arises when a node in the cloth hits the ball.

The problem setting



$t = t_2$



- The cloth is allowed to fall under gravity until it collides with the ball ($t=t_1$)
- You follow the motion of the nodes using the same velocity verlet algorithm that you used for the MD

$$y_i(t_k) = y_i(t_{k-1}) + v_i^y(t_{k-1})\Delta t + \frac{1}{2} \frac{F_i^y(t_{k-1})}{m_{node}} \Delta t^2$$

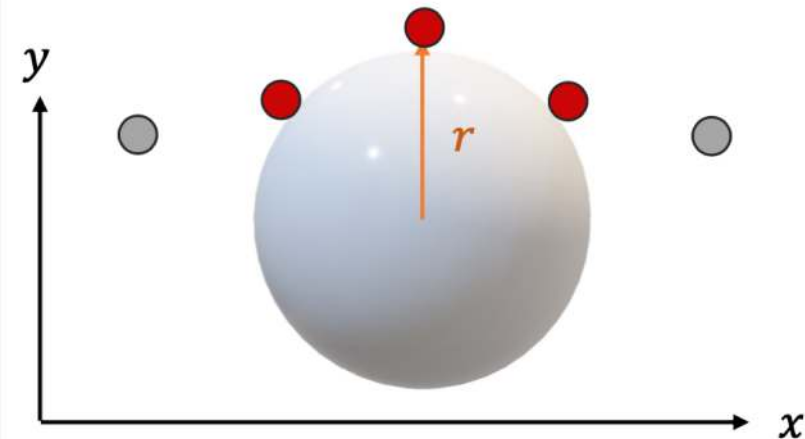
$$v_i^y(t_k) = v_i^y(t_{k-1}) + \frac{1}{2} \frac{F_i^y(t_k) + F_i^y(t_{k-1})}{m_{node}} \Delta t$$

- The difference arises when a node in the cloth hits the ball.
- You detect this by noticing that the updated position of the node is within the radius of the ball. At this point you move the node to the nearest point on the surface of the ball.

The problem setting



$t = t_2$



- The cloth is allowed to fall under gravity until it collides with the ball ($t=t_1$)
- You follow the motion of the nodes using the same velocity verlet algorithm that you used for the MD

$$y_i(t_k) = y_i(t_{k-1}) + v_i^y(t_{k-1})\Delta t + \frac{1}{2} \frac{F_i^x(t_{k-1})}{m_{node}} \Delta t^2$$
$$v_i^y(t_k) = v_i^y(t_{k-1}) + \frac{1}{2} \frac{F_i^x(t_k) + F_i^x(t_{k-1})}{m_{node}} \Delta t$$

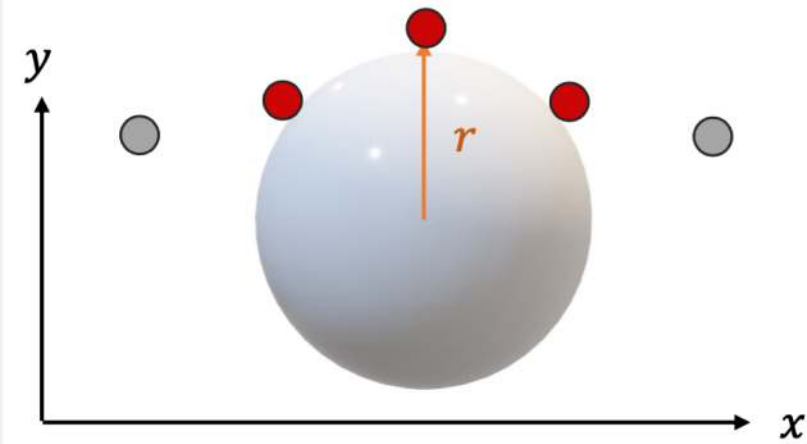
- The difference arises when a node in the cloth hits the ball.
- You detect this by noticing that the updated position of the node is within the radius of the ball.
- At this point you move the node to the nearest point on the surface of the ball.

The problem setting



$$t = t_2$$

- What about the velocity?

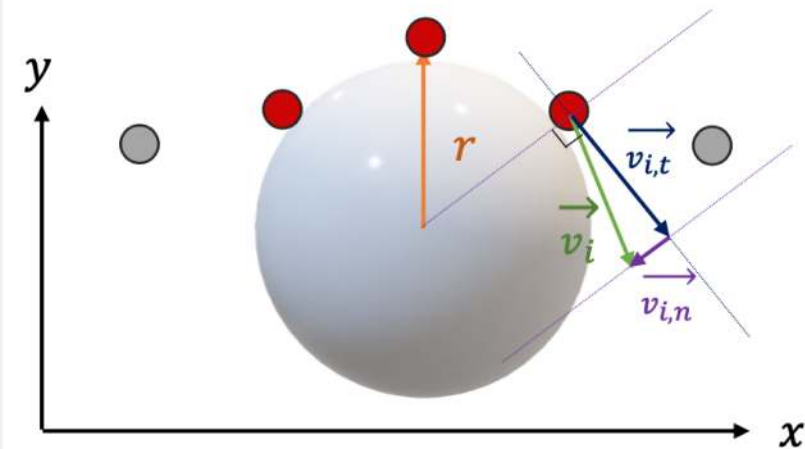


The problem setting



$$t = t_2$$

- What about the velocity?
- We set to zero the module of the component of the velocity normal to the surface of the sphere, $v_{i,n}$.



The python code



```

176 iter = 0
177 maxit = 400
178 while iter < maxit:
179     iter += 1
180     if verbose:
181         print("iteration and potential energy ", iter, PE)
182
183     # Update coordinates using same MD velocity verlet
184     for node in nodes:
185         node.pos += dt * (node.velocity + dt * node.force * 0.5)
186         node.oldforce = node.force
187
188     # apply constraints (move nodes to surface of ball)
189     for node in nodes:
190         dist = node.pos - vector(myball.x, myball.y, myball.z)
191         if dist.mag < myball.radius:
192             fvector = dist / dist.mag * myball.radius
193             node.pos = vector(myball.x, myball.y, myball.z) + fvector
194
195     if iter % update == 0:
196         # update the view if necessary
197         for nx in range(N - 1):
198             for ny in range(N - 1):
199                 nodes[nx * N + ny].fill.v0.pos = nodes[nx * N + ny].pos
200                 nodes[nx * N + ny].fill.v1.pos = nodes[(nx + 1) * N + ny].pos
201                 nodes[nx * N + ny].fill.v2.pos = nodes[(nx + 1) * N + ny + 1].pos
202                 nodes[nx * N + ny].fill.v3.pos = nodes[nx * N + ny + 1].pos
203
204     sleep(0.05)
205     PE = compute_force(interact, gravity, separation, fcon)
206
207     # Update velocity using same MD velocity verlet
208     damp = 0.995
209     for node in nodes:
210         node.velocity += dt * (node.force + node.oldforce) * 0.5 * damp

```

- Lines 184-186 update the positions of the nodes

$$y_i(t_k) = y_i(t_{k-1}) + v_i^y(t_{k-1})\Delta t + \frac{1}{2} \frac{F_i^x(t_{k-1})}{m_{node}} \Delta t^2$$

- Lines 189-193 apply constraints, e.g. move the node to the nearest point on the surface of the ball
- Line 195-202 update the visualization panel
- Line 205 computes the forces and the potential energy

$$F_{ij} = -k \left(\vec{R}_{ij} - \vec{R}_{ij}^{eq} \right), \vec{F}_i = \vec{G} + \sum_{\substack{j \neq i: \\ X_i - \delta \leq X_j \leq X_i + \delta \\ Y_i - \delta \leq Y_j \leq Y_i + \delta}} \vec{F}_{ij}$$

- Lines 208-210 update the velocity

$$v_i^y(t_k) = v_i^y(t_{k-1}) + \frac{1}{2} \frac{F_i^x(t_k) + F_i^x(t_{k-1})}{m_{node}} \Delta t$$

The python code: collision correction



```
176 iter = 0
177 maxit = 400
178 while iter < maxit:
179     iter += 1
180     if verbose:
181         print("iteration and potential energy ", iter, PE)
182
183     # Update coordinates using same MD velocity verlet
184     for node in nodes:
185         node.pos += dt * (node.velocity + dt * node.force * 0.5)
186         node.oldforce = node.force
187
188     # apply constraints (move nodes to surface of ball)
189     for node in nodes:
190         dist = node.pos - vector(myball.x, myball.y, myball.z)
191         if dist.mag < myball.radius:
192             fvector = dist / dist.mag * myball.radius
193             node.pos = vector(myball.x, myball.y, myball.z) + fvector
194
195     if iter % update == 0:
196         # update the view if necessary
197         for nx in range(N - 1):
198             for ny in range(N - 1):
199                 nodes[nx * N + ny].fill.v0.pos = nodes[nx * N + ny].pos
200                 nodes[nx * N + ny].fill.v1.pos = nodes[(nx + 1) * N + ny].pos
201                 nodes[nx * N + ny].fill.v2.pos = nodes[(nx + 1) * N + ny + 1].pos
202                 nodes[nx * N + ny].fill.v3.pos = nodes[nx * N + ny + 1].pos
203
204         sleep(0.05)
205         PE = compute_force(interact, gravity, separation, fcon)
206
207     # Update velocity using same MD velocity verlet
208     damp = 0.995
209     for node in nodes:
210         node.velocity += dt * (node.force + node.oldforce) * 0.5 * damp
```

- Lines 189-193 : if the new coordinates for the cloth are within the ball, then that node of the cloth is moved back to the surface of the ball.
- Problem: the velocity of the node remains the same.
- Only the component of the velocity that is tangential to the surface is allowed to be non-zero following a collision of a cloth node with the ball.
- You need to modify to correctly update the velocities of the nodes.



Short demonstration of the python code
(watch video recording on Echo360)



Short discussion and demonstration on the C/C++ code
(**watch video recording on Echo360**)

Assignment 2 – Structure

Step 1: Collision Correction (no marks associated with it)

- assist you in understanding the numerical methods

Step 2: Performance Assessment

- OpenGL version for debugging
- Kernel version for performance testing on Gadi

Step 3: SSE/AVX and OpenMP Vectorization

- Vectorize and compare performance

Step 4: Parallelization using OpenMP

- Parallelize using OpenMP and compare performance

Step 5: Roofline analysis

- Produce and analyse a roofline plot

Step 6: Block-wise decomposition (COMP6464 only)

- Split both columns and rows among available threads

Assignment 2 – Important Information

- Assignment 2 is available now on GitLab
<https://gitlab.cecs.anu.edu.au/comp3320/2024/comp3320-2024-assignment-2>
- Fork the repository to get started!
- Due at **5pm on October 23 (Wednesday)!**
- Worth **20% of final mark**
- **VERY IMPORTANT: Start working on the assignment asap!**
You are unlikely to complete the assignment satisfactorily in less than 3 weeks.
- Push your submission including all necessary files by the deadline or you will receive late penalties.
- **Late penalty of 5% of the awarded mark for each working day late or part thereof (as per course administrative handout).**